



Last Modified: Wed, 08 Nov 2023 02:25:44 GMT

Pulse Width Modulation and Hardware Timers in zeptoforth on the Raspberry Pi Pico - tabemann/zeptoforth GitHub Wiki

Content mirrored for search engine indexing from:

<https://github.com/tabemann/zeptoforth/wiki/Pulse-Width-Modulation...>

Why does this service exist?

Note that this functionality is divided up into two separate peripherals on the RP2040, whereas it is often unified on other microcontrollers, such as STM32 microcontrollers.

Pulse Width Modulation

Note: below all words referenced when not otherwise obvious should be assumed to be in the `pwm` module.

The PWM peripheral for the RP2040 can both generate PWM output signals and act as a counter for input signals. It is divided up into eight "slices", each of which correspond to two pairs of A pins and B pins (except for slice seven which only corresponds to one pair). Each slice maintains its own counter (set with `pwm-counter!` and read with `pwm-counter@`), its own "top" value for its counter (set with `pwm-top!`), its own clock divider relative to the system clock (set with `pwm-clock-div!`), its own setting as to how to treat inputs into its B pin, its own phase correct flag (set with `pwm-phase-correct!`), and its own interrupt enable and interrupt status flags. Also, the A pins and B pins have separate compare values (set with `pwm-counter-compare-a!` and `pwm-counter-compare-b!`) and invert flags (set with `pwm-invert-a!` and `pwm-invert-b!`). All slices share a single PWM interrupt, so if one is handling multiple slices' interrupts, the PWM interrupt handler must separately check each slice in question's interrupt status flag.

When a pin for a slice is outputting and phase-correct modulation is not enabled, when the slice's counter starts at or wraps around to zero, the pin goes high, and then when the slice's counter reaches the pin's compare value, the pin goes low until the next time the slice's counter reaches the slice's top value. Phase-correct modulation modifies this so that when a

slice's counter reaches the slice's top value it counts downward until it reaches zero, staying low until it passes the slice's compare value, where then the pin goes high again; this has the effect that the phase of the signal is independent of the pin's compare value and hence its duty cycle.

There are four modes in which the slice's counter may operate. One is free-running mode, where it increases by one for each cycle generated by the slice's clock divider. Another is gated mode, where it increases by one for each cycle generated by the slice's clock divider while the slice's pin B is input high. Another is rising edge mode, where it increases by one for each rising edge detected on the slice's pin B. And last but not least is falling edge mode, where it decreases by one for each falling edge detected on the slice's pin B. It should be noted that outside of free-running mode the compare value of the slice's pin B is ignored; only in free-running mode does pin B output.

Interrupts are signaled for slices for which interrupts have been enabled whenever the counter for the slice in question reaches zero, whether through wrapping around in non-phase-correct mode or through counting downwards to zero in phase-correct mode. There is one PWM interrupt handler, which is to be set with `pwm-vector!`. The interrupt state of each PWM slice can be gotten through calling `pwm-int@`, which returns a bitmask with each of the eight lowest bits corresponding to the interrupt state of a given slice. The PWM interrupt handler must then call `clear-pwm-int` with a bitmask of each of the interrupts that it is to clear. To clear all interrupts, use `pwm-int@ clear-pwm-int`. Afterwards, call `clear-pwm-pending` before leaving the PWM interrupt handler to clear the IRQ pending state.

The slices of the PWM peripheral are mapped such that for GPIO pins 0 through 15 slice zero corresponds to the pair of GPIO pin 0 and GPIO pin 1 as A and B pins, through slice seven which corresponds to the pair of GPIO pin 14 and GPIO pin 15 as A and B pins, and then for GPIO pins 16 through 29 slice zero also corresponds to the pair of GPIO pin 16 and GPIO pin 17 as A and B pins, through slice six which also corresponds to the pair of GPIO pin 28 and GPIO pin 29 as A and B pins.

Example

Here is an example where we connect one PWM slice's pin A (GPIO pin 6 of PWM slice 3) to another PWM slice's pin B (GPIO pin 9 of PWM slice 4) and we generate pulses in the first PWM slice's pin A and count the rising edges received on the other PWM slice's B.

Here is our imports and our constants:

```
pwm import
pin import

\ PWM output index
3 constant pwm-out-index

\ PWM input index
4 constant pwm-in-index

\ PWM output GPIO pin
6 constant pwm-out-pin

\ PWM input GPIO pin
9 constant pwm-in-pin

\ PWM output wrap
3 constant pwm-out-wrap
```

```
\ PWM input wrap
65535 constant pwm-in-wrap
```

```
\ PWM output compare value
2 constant pwm-out-compare
```

Here is our counter for counting pulses as a double cell. Note that we do not count individual pulses but rather we count the total number of pulses for each time that the PWM slice 4's counter wraps, triggering a PWM interrupt

```
\ Our counter
2variable counter
```

Here is our PWM interrupt handler. As we are only raising PWM interrupts for one PWM slice we do not check for the individual slices. Each time this is called we increment counter by `pwm-in-wrap`, because this PWM interrupt is called for `pwm-in-wrap` intervals; there is no need here to check the actual state of the PWM slice 4's counter. When we are done we clear the PWM interrupt state.

```
\ Our interrupt handler
: handle-pwm ( -- )
  pwm-in-wrap s>d counter 2+!
  pwm-in-index bit clear-pwm-int
  clear-pwm-pending
;
```

Here we set up and run our test:

```
\ Run our test
: run-test ( -- )
```

This is basic pin setup:

```
pwm-out-pin pull-up-pin
pwm-out-pin fast-pin
pwm-out-pin pwm-pin
pwm-in-pin pull-up-pin
pwm-in-pin fast-pin
pwm-in-pin pwm-pin
```

Here we set up the vector for the PWM interrupt:

```
['] handle-pwm pwm-vector!
```

Here we disable PWM slices 3 and 4, and disable interrupts for PWM slice 4:

```
pwm-out-index bit pwm-in-index bit or disable-pwm
pwm-in-index bit disable-pwm-int
```

Here we initialize our counter, and the counters for both PWM slices 3 and 4:

```
0. counter 2!
0 pwm-out-index pwm-counter!
0 pwm-in-index pwm-counter!
```

Here we initialize the "top" values at which each PWM slice wraps; we also initialize the pin A compare value for PWM slice 3 (we have no need to do so for pin B of PWM slice 4 as inputs ignore their compare values):

```
pwm-out-wrap pwm-out-index pwm-top!
pwm-in-wrap pwm-in-index pwm-top!
```

```
pwm-out-compare pwm-out-index pwm-counter-compare-a!
```

Here we set PWM slice 3 to have a free-running counter, incrementing once for each cycle, so as to generate a continuous sequence of pulses, while we set PWM slice 4 to increment its counter once for each input rising edge:

```
pwm-out-index free-running-pwm  
pwm-in-index rising-edge-pwm
```

Here we set the clock divider for PWM slice 3 to 16, so as to have a clock of a 125 MHz / 16 = 7.8125 MHz rate, and set the clock divider for PWM slice 4 to 1, so to have a clock of 125 MHz:

```
0 16 pwm-out-index pwm-clock-div!  
0 1 pwm-in-index pwm-clock-div!
```

Here we enable PWM interrupts for PWM slice 4 and then enable PWM slices 3 and 4 simultaneously.

```
pwm-in-index bit enable-pwm-int  
pwm-out-index bit pwm-in-index bit or enable-pwm
```

Last but not least, every 500 ms we check the counter incremented by the PWM interrupt handler and display its value until the user presses a key:

```
begin key? not while  
  cr counter 2@ d.  
  500 ms  
repeat  
key drop  
;
```

Hardware Timer

Note: below all words referenced when not otherwise obvious should be assumed to be in the timer module.

Technically speaking, the RP2040 has a total of *ten* hardware counters, one being the hardware timer peripheral's counter, one being the ARM Cortex-M0+ core's SysTick, and eight being the counters for each PWM slice. However, here we will be focused on the hardware timer peripheral. The hardware timer peripheral has one 64-bit counter that operates, by default, at a 1 MHz rate. It requires the watchdog timer to be operative, as it shares its functionality with it; this is operative from bootup in zeptoforth. As the 64-bit counter can represent a time at a 1 MHz resolution for thousands of years, it can be practically treated as monotonic, even though functionality does exist to change the time with `us-counter!`, or to pause and un-pause it with `pause-us` and `unpause-us` respectively, if one so desires (note that this is recommended in most applications). The 64-bit counter can be read with `us-counter`; note that even though multicore/interrupt-unsafe means of reading the hardware timer peripheral's counter which use latching to get more accurate times are provided by the RP2040, `us-counter` does not make use of these, and rather checks to ensure that for two LSB reads, with an MSB read in between, the time has not changed, and hence the time can be treated as accurate, providing no sets to the time with `us-counter!`.

The hardware timer peripheral has four "alarms" that can be set with `set-alarm` which trigger interrupts when the hardware timer's counter lower 32 bits reaches the 32-bit value

set for the alarms. These alarms are on a one-time basis; for each time they are triggered, the user must reset them manually if they wish to have them trigger in the future. Note, however, that the user must clear the interrupt state for the alarm in question with `clear-alarm-int` within the interrupt handler for the alarm or else the alarm will trigger indefinitely. Additionally, set alarms can be disarmed with `clear-alarm`.

There are also blocking waits for 64-bit time values and intervals, enabled by using `delay-until-us` and `delay-us` respectively. Note that these do not relinquish control to other tasks, but unless interrupts are disabled may be preempted. They are only recommended for short, high resolution delays, and in this application it may be permissible to disable interrupts during the delay to prevent preemption.

Examples

Simple uses of `delay-us` and `delay-until-us`

Here we have two simple use cases for `delay-us` and `delay-until-us`. They are just to illustrate their use, and are not recommended practical usages, because of the combination of the very long delays (500 ms) combined with the blocking nature of `delay-us` and `delay-until-us`.

Here we import the timer module:

```
timer import
```

Here we define our delay duration, 500,000 microseconds. This is not a practical duration for blocking waits but rather to provide a user-visible delay time:

```
\ Delay duration
500000. 2constant delay-interval
```

Here we define `display-asterisks`, which displays asterisks on the console at 500,000 microsecond intervals. Note that because we are waiting from the present time, the timing of the displayed asterisks will drift over time:

```
\ Display 25 asterisks at intervals
: display-asterisks ( -- )
  25 0 do delay-interval delay-us ." *" loop
;
```

Here we define `display-accurate-pluses`, which displays pluses on the console at 500,000 microsecond intervals. Note that because we are waiting until a time defined with the local variables `next-delay-lo` and `next-delay-hi`, which we increment by the set interval for each cycle, we get a more accurate timing that will not drift:

```
\ Display 25 pluses at accurate intervals
: display-accurate-pluses ( -- )
  us-counter delay-interval d+ { next-delay-lo next-delay-hi }
  25 0 do
    next-delay-lo next-delay-hi delay-until-us ." +"
    next-delay-lo next-delay-hi delay-interval d+ to next-delay-hi to next-delay-lo
  loop
;
```

And here is what we get when we run the above:

```
display-asterisks ***** ok
display-accurate-pluses ++++++ ok
```

In both of these cases, the characters printed on the console are roughly spaced by 500,000 microseconds, but in the latter case the timing will not drift over time.

A combined hardware timer and PWM shaded LED example

Here we import the pwm and timer modules and set up some control variables:

```
pwm import
timer import

\ The blinker maximum input shade
variable shade-max-input-shade

\ The blinker maximum shade
variable shade-max-shade

\ The blinker shading
variable shade-shade

\ The blinker shade step delay in 100 us increments
variable shade-step-delay

\ The blinker multiplier
2variable shade-multiply

\ The blinker pre-multiplier
2variable shade-premultiply

\ Shade increment
variable shade-increment

\ Shade level
variable shade-level

\ Maximum shade level
variable max-shade-level

\ Alarm interval
variable alarm-interval
```

Here we select the PWM slice we are to use; as we are controlling GPIO pin 25, as we are controlling the Raspberry Pi Pico's onboard LED, we need to use PWM slice 4:

```
\ PWM slice
4 constant pwm-slice
```

The following word, along with the control variables shade-max-input-shade, shade-premultiply, and shade-multiply, defines the relationship between the linear shading input value and the actual duty cycle of the LED:

```
\ The blinker shade conversion routine
: convert-shade ( i -- shade )
  s>f shade-max-input-shade @ s>f f/ pi f* cos dnegate 1,0 d+ 2,0 f/
  shade-premultiply 2@ f* expm1 shade-premultiply 2@ expm1 f/
  shade-multiply 2@ f* f>s
;
```

Here is our alarm 0 interrupt handler:

```
\ Alarm handler
defer handle-alarm
:noname ( -- )
```

First thing we do in it is clear the alarm 0 interrupt state.

```
0 clear-alarm-int
```

We execute the rest of the alarm 0 interrupt handler in a software exception handler because, if the control variables are misconfigured, `pwm-counter-compare-b!` may raise an exception, and uncaught software exceptions within interrupt handler will result in a system crash.

```
[:
```

Here we increment or decrement `shade-level`, alternating direction when `shade-level` reaches 0 or `shade-max-input-shade`:

```
shade-level @ 0<= if
  shade-increment @ abs shade-increment !
else
  shade-level @ shade-max-input-shade @ >= if
    shade-increment @ abs negate shade-increment !
  then
then
shade-increment @ shade-level +!
```

Here we convert `shade-level` into a duty cycle value from 0 to `max-shade-level`, i.e. a duty cycle of 100%, with `convert-shade` and set the PWM slice compare value for pin B for PWM slice 4 to this value:

```
shade-level @ convert-shade pwm-slice pwm-counter-compare-b!
```

Here we actually catch any exceptions and execute them, to display a message, without causing any unnecessary problems:

```
;] try ?execute
```

Here we reset alarm 0 for `handle-alarm` at `us-counter-lsb` plus `alarm-interval`:

```
us-counter-lsb alarm-interval @ + ['] handle-alarm 0 set-alarm
; ' handle-alarm defer!
```

Here we run our LED shader:

```
\ Shade an LED
: run-shade-led ( -- )
```

Here we disable PWM slice 4:

```
pwm-slice bit disable-pwm
```

Here we initialize the control variables used by the LED shader; note in particular that we are setting the interval for alarm 0 to 2500 microseconds:

```
125 shade-max-input-shade !
125,0 shade-multiply 2!
15,0 shade-premultiply 2!
2500 alarm-interval !
```

```
0 shade-level !
1 shade-increment !
```

Here we calculate max-shade-value based on shade-max-input-shade converted by convert-shade to define the high end of the range of the duty cycle, i.e. a PWM top value of max-shade-level will equal a duty cycle of 100%.

```
shade-max-input-shade @ convert-shade max-shade-level !
```

Here we set the GPIO pin 25, i.e. the Raspberry Pi Pico's onboard LED, to be an PWM pin:

```
25 pwm-pin
```

Here we initialize the counter value for PWM slice 4 to 0 and the top value for PWM slice 4 to max-shade-level:

```
0 pwm-slice pwm-counter!
max-shade-level @ pwm-slice pwm-top!
```

Here we initialize the clock divider of PWM slice 4 to 255, i.e. that it would have a clock of about 490 kHz:

```
0 255 pwm-slice pwm-clock-div!
```

Here we configure PWM slice 4 to use phase-correct modulation:

```
true pwm-slice pwm-phase-correct!
```

Here we enable PWM slice 4:

```
pwm-slice bit enable-pwm
```

Here we set alarm 0 for handle-alarm to the lower 32 bits of the hardware timer's counter plus the alarm interval of 2500 microseconds.

```
us-counter-lsb alarm-interval @ + ['] handle-alarm 0 set-alarm
;
```

In all, this produces a shaded blinky which (quickly) fades the LED on the Raspberry Pi Pico on and off rather than merely turning on and off. Of course, the LED does turn on and off, but it turns on and off so fast that to the human eye it appears to vary in intensity.

 [Page Index for this GitHub Wiki](#)